Name: Dhruv Makwana

Module 17) Rest Framework

1.1) What is an API (Application Programming Interface)?

Ans: API is a set of rules that allows communication between two applications.
It lets software exchange data using requests and responses.
Example: A mobile app fetching data from a server.

1.2) Types of APIs: REST, SOAP.

Ans: REST is lightweight, uses JSON, and is widely used in web apps.
SOAP is protocol-based, uses XML, and is more secure but complex.
REST is faster and more flexible than SOAP.

1.3) Why are APIs important in web development?

Ans: APIs enable communication between frontend and backend.
They allow integration with third-party services.
Used for building scalable and modular applications.

1.4) Understanding project requirements.

Ans: Analyze what features and data the API should provide.
Identify endpoints, inputs, and outputs.
Helps in designing efficient APIs.

1.5) Setting up the environment and installing necessary packages.

Ans: Create virtual environment using venv.
Install Django and DRF using pip.
Example: pip install djangorestframework.

1.6) What is Serialization?

Ans: Serialization converts complex data (models) into JSON format.
It helps send data over APIs.
Deserialization converts JSON back to Python objects.

1.7) Converting Django QuerySets to JSON.

Ans: Use serializers or JsonResponse.
QuerySets are not directly JSON serializable.
Example: serializer = MySerializer(queryset, many=True).

1.8) Using serializers in Django REST Framework (DRF).

Ans: Serializers convert model data into JSON.
Defined in serializers.py.
Used for validation and data transformation.

1.9) HTTP request methods (GET, POST, PUT, DELETE).

Ans: GET: Retrieve data.
POST: Create new data.
PUT: Update data.
DELETE: Remove data.

1.10) Sending and receiving responses in DRF.

Ans: Use Response() to send data in DRF.
Handles JSON conversion automatically.
Example: return Response(serializer.data).

1.11) Understanding views in DRF: Function-based views vs Class-based views.

Ans: Function-based views are simple and easy.
Class-based views are reusable and organized.
DRF also provides generic views for CRUD.

1.12) Defining URLs and linking them to views.

Ans: Use urls.py to map endpoints.
Example: path('api/', views.api_view).
Connects client requests to logic.

1.13) Adding pagination to APIs to handle large data sets.

Ans: Pagination limits number of records per request.
Improves performance for large data.
Configured in settings.py or views.

1.14) Configuring Django settings for database, static files, and API keys.

Ans: Set database, static files, and API keys in settings.py.
Ensure proper security configurations.
Used to manage project environment.

1.15) Setting up a Django REST Framework project.

Ans: Install DRF and add to INSTALLED_APPS.
Create serializers, views, and URLs.
Run server to test APIs.

1.16) Implementing social authentication (e.g., Google, Facebook) in Django.

Ans: Use OAuth2 with packages like django-allauth.
Register app with providers like Google or Facebook.
Get API keys and configure settings.

1.17) Sending emails and OTPs using third-party APIs like Twilio, SendGrid.

Ans: Use services like Twilio or SendGrid.
Send emails or SMS using API calls.
Useful for verification systems.

1.18) REST principles: statelessness, resource-based URLs, and using HTTP methods for CRUD operations.

Ans: REST APIs are stateless (no session storage).
Use resource-based URLs like /users/1.
HTTP methods perform CRUD operations.

1.19) What is CRUD, and why is it fundamental to backend development?

Ans: CRUD = Create, Read, Update, Delete.
Basic operations for database handling.
Core concept in backend development.

1.20) Difference between authentication and authorization.

Ans: Authentication verifies user identity (login).
Authorization checks permissions (access rights).
Example: Admin vs normal user access.

1.21) Implementing authentication using Django REST Framework's token-based system.

Ans: DRF provides token authentication system.
User gets token after login.
Token is used in headers for API access.

1.22) Introduction to OpenWeatherMap API and how to retrieve weather data.

Ans: OpenWeatherMap API provides weather data.
Use API key to fetch temperature, humidity.
Returns JSON response.

1.23) Using Google Maps Geocoding API to convert addresses into coordinates.

Ans: Converts address into latitude and longitude.
Provided by Google Maps Geocoding API.
Useful for location-based apps.

1.24) Introduction to GitHub API and how to interact with repositories, pull requests, and issues.

Ans: GitHub API allows access to repos, issues, PRs.
Used to automate tasks.
Returns JSON data.

1.25) Using Twitter API to fetch and post tweets, and retrieve user data.

Ans: Twitter API allows posting tweets and fetching data.
Requires authentication keys.
Used for social media integration.

1.26) Introduction to REST Countries API and how to retrieve country-specific data.

Ans: REST Countries API provides country details.
Includes population, capital, currency.
Useful for global apps.

1.27) Using email sending APIs like SendGrid and Mailchimp to send transactional emails.

Ans: SendGrid and Mailchimp send emails via API.
Used for notifications and marketing.
Supports templates and automation.

1.28) Introduction to Twilio API for sending SMS and OTPs.

Ans: Twilio provides SMS services.
Used for OTP verification.
Easy integration with Django.

1.29) Introduction to integrating payment gateways like PayPal and Stripe.

Ans: PayPal and Stripe handle online payments.
Provide secure APIs.
Used in e-commerce apps.

1.30) Using Google Maps API to display maps and calculate distances between locations.

Ans: Google Maps API displays maps and calculates distances.
Used for navigation and tracking.
Integrated using JavaScript and API keys.